

Characterizing and Optimizing LLM Post-Training Efficiency on Consumer AMD RDNA3 GPUs with Unsloth

Hongyu Chen

<https://github.com/ChenHongYu2026>

DOI: [10.5281/zenodo.21695686](https://doi.org/10.5281/zenodo.21695686) Code: github.com/ChenHongYu2026/llm-training-on-consumer-amd

Abstract

Consumer-grade AMD GPUs lack dedicated tensor cores, yet serve a growing community of LLM practitioners. We present a systematic efficiency characterization of LLM post-training (SFT and GRPO) on an AMD RX 7900 XTX (RDNA3) using the Unsloth framework. We propose a *dual-peak MFU methodology* that reports utilization against both the theoretical spec-sheet peak (122.8 TFLOPS) and a measured GEMM practical peak (101.53 TFLOPS), revealing a 17.3% systematic bias in conventional reporting. Through component-stacking gap attribution, we decompose the 97.7% efficiency gap from GEMM peak to full GRPO training into interpretable stages, showing that autoregressive generation dominates the end-to-end loss. Framework comparison across 3 seeds shows Unsloth BF16 achieves 5.2% higher throughput than HuggingFace native (129 vs 123 tok/s, steady-state corrected caliber), while 4-bit quantization is net-negative on RDNA3 (−40.7%). A systematic hyperparameter sweep identifies an optimal recipe (G=2, batch=2×4, seq=256) achieving 179 tok/s (+36% over defaults) at 3.3% practical MFU. All measurements pass a formal validity protocol (grad_norm>0) and a post-hoc audit that corrected a profiler token-accounting bug (cumulative-count inflation up to 10.6×), ensuring all reported throughput reflects genuine training under a verified measurement caliber.

1 Introduction

The proliferation of large language models (LLMs) has democratized AI capabilities, yet the computational cost of post-training, including supervised fine-tuning (SFT) and reinforcement learning from human feedback (RLHF/GRPO), remains a barrier for individual researchers and small teams. Consumer-grade GPUs, particularly AMD’s RDNA3 architecture (RX 7900 XTX, 24GB), offer an accessible alternative to expensive data-center hardware. However, unlike NVIDIA’s ecosystem with extensive efficiency studies on A100/H100 clusters (Chowdhery et al., 2022; Hoffmann et al., 2022), the training efficiency characteristics of consumer AMD GPUs remain largely unexplored in academic literature.

Existing MFU (Model FLOPS Utilization) studies universally assume dedicated tensor cores (NVIDIA Tensor Cores, AMD Matrix Cores on Instinct). RDNA3 consumer GPUs implement matrix operations via WMMA (Wave Matrix Multiply-Accumulate) microcoded vector instructions rather than dedicated silicon, making the spec-sheet peak (122.8 TFLOPS BF16) fundamentally unreachable in practice. This architectural distinction necessitates a new measurement methodology, our *dual-peak MFU* approach, that separately reports utilization against theoretical and practical peaks.

Unsloth (Unsloth AI, 2024) is a widely adopted training framework that recently introduced official AMD support in collaboration with the AMD team, reporting up to 2× speedups on data-center Instinct GPUs equipped with dedicated Matrix Cores (CDNA architecture). However, consumer RDNA 3 lacks any dedicated AI acceleration hardware and relies entirely on WMMA microcoded vector instructions, making efficiency characteristics observed on CDNA-class hardware non-transferable.

Contributions. We make the following contributions:

1. A **dual-peak MFU reporting methodology** that separately measures utilization against the spec-sheet theoretical peak and a practical GEMM peak, with quantified bias for architectures lacking dedicated tensor cores.
2. A **systematic efficiency characterization** of LLM post-training on consumer RDNA3, including framework comparison (Unsloth vs HF, 3 seeds) and component-stacking gap attribution revealing a 75.7% efficiency gap.
3. A **tuning study** across 10 configurations with mechanistic explanations, yielding an actionable optimal recipe for practitioners.
4. An **open-source profiler** incorporating a training validity check (`grad_norm>0`) to ensure reported throughput reflects genuine parameter updates.

2 Background & Related Work

2.1 RDNA3 Architecture and MFU Challenges

AMD’s RDNA3 architecture (gfx1100) implements matrix operations through WMMA instructions that operate on wavefront-level (32/64 threads) vector registers, unlike NVIDIA’s dedicated Tensor Cores which execute matrix multiply-accumulate in dedicated silicon. This microcoded approach means the advertised 122.8 TFLOPS BF16 peak represents a theoretical upper bound achievable only under ideal instruction scheduling; realistic GEMM workloads achieve 82.7% of this peak (101.53 TFLOPS in our measurement). The 960 GB/s GDDR6 bandwidth (vs 2–3 TB/s HBM on NVIDIA data-center cards or 5.3–8 TB/s on AMD Instinct MI300X/MI355X) further constrains memory-bound operations ([Advanced Micro Devices, 2022](#)).

2.2 LLM Training Efficiency Studies

Model FLOPS Utilization (MFU), defined as the ratio of achieved FLOPS to hardware peak, has become the standard efficiency metric for LLM training. The $6ND$ approximation (N =parameters, D =tokens) provides a tractable FLOPS estimate ([Hoffmann et al., 2022](#); [Kaplan et al., 2020](#)). PaLM ([Chowdhery et al., 2022](#)) established a pretraining baseline of 46–57% MFU on TPU v4, yet RL-based post-training appears far less efficient: recent GRPO reports indicate only 10.5% MFU on multi-GPU A100 clusters (SemiAnalysis).

2.3 Unsloth and Consumer-Grade Fine-tuning

Unsloth ([Unsloth AI, 2024](#)) claims $2\times$ speedups through kernel fusion and optimized memory access patterns, but its benchmarks are marketing-oriented without peer review. Unsloth recently introduced official AMD support in collaboration with the AMD team, with reported $2\times$ speedups on data-center Instinct GPUs (CDNA architecture); on consumer RDNA3 hardware, the execution path relies on ROCm’s HIP/Triton backends ([Advanced Micro Devices, 2024](#)). LlamaFactory ([Zheng et al., 2024](#)) provides another popular framework but its ROCm compatibility remains unverified for consumer GPUs. TRL ([von Werra et al., 2022](#)) and DeepSpeed ([Rasley et al., 2020](#)) offer additional training infrastructure. To our knowledge, no academic study has rigorously characterized training efficiency on consumer AMD hardware.

3 Methodology

3.1 Experimental Design Overview

We investigate three research questions through a progressive experimental pipeline:

- **RQ1 (Framework Characterization):** How do training frameworks compare in efficiency on consumer RDNA3?
- **RQ2 (Gap Attribution):** Where does the efficiency gap between hardware peak and real training originate?
- **RQ3 (Tuning Optimization):** Which hyperparameters most impact throughput, and what is the optimal configuration?

The pipeline proceeds in three phases: Phase B1 addresses RQ1 via controlled framework comparison (Unsloth vs. HuggingFace, 3 seeds each); Phase B2 addresses RQ2 via component-stacking gap attribution (L0–L4); Phase C addresses RQ3 via a 10-configuration hyperparameter sweep. All phases share the same workload: GSM8K GRPO training on Qwen2.5-3B-Instruct (3.09B parameters). We select this workload because (1) GRPO combines autoregressive generation with gradient updates, exercising both bandwidth-bound and compute-bound regimes; (2) GSM8K’s rule-based reward (correctness verification) eliminates LLM-judge variability; (3) the 3B scale fits within 24 GB consumer VRAM while remaining architecturally representative.

3.2 Dual-Peak MFU Measurement

Motivation. RDNA3 implements matrix operations via WMMA microcoded vector instructions rather than dedicated tensor-core silicon. Unlike NVIDIA architectures where $P_{\text{GEMM}}/P_{\text{spec}} > 0.95$, this microcoded path cannot saturate the advertised peak under any workload. Reporting MFU solely against the spec-sheet value therefore introduces a fixed architectural bias that conflates hardware limitation with software inefficiency. To disentangle the two, we measure a *practical peak* via a standalone GEMM benchmark and report dual MFU metrics.

GEMM benchmark. Listing 1 shows the measurement kernel. We multiply two 8192×8192 BF16 matrices for 50 timed iterations (after 10 warmup), with explicit device synchronization to exclude host-side latency.

```
1 def measure_gemm_peak(size=8192, warmup=10, iters=50):
2     a = torch.randn(size, size, dtype=torch.bfloat16,
3                     device="cuda")
4     b = torch.randn(size, size, dtype=torch.bfloat16,
5                     device="cuda")
6     for _ in range(warmup):
7         torch.matmul(a, b)
8     torch.cuda.synchronize()
9     t0 = time.perf_counter()
10    for _ in range(iters):
11        torch.matmul(a, b)
12    torch.cuda.synchronize()
13    elapsed = time.perf_counter() - t0
14    tflops = 2.0 * size**3 * iters / elapsed / 1e12
15    return tflops # 101.53 (82.7% of 122.8 spec)
```

Listing 1: Practical GEMM peak measurement (BF16, 8192×8192).

This yields $P_{\text{GEMM}} = 101.53$ TFLOPS, i.e., 82.7% of the 122.8 TFLOPS spec-sheet value.

MFU definitions. We define two complementary metrics:

$$\text{MFU}_{\text{theo}} = \frac{6ND/\Delta t}{P_{\text{spec}}}, \quad \text{MFU}_{\text{prac}} = \frac{6ND/\Delta t}{P_{\text{GEMM}}} \quad (1)$$

where N is total model parameters, D is tokens processed in time Δt , $P_{\text{spec}} = 122.8$ TFLOPS, and $P_{\text{GEMM}} = 101.53$ TFLOPS. The systematic bias of reporting only MFU_{theo} is $1 - P_{\text{GEMM}}/P_{\text{spec}} \approx 17.3\%$. We recommend dual-peak reporting for any architecture where this ratio falls below 0.95.

FLOPS estimation. Training-step FLOPS use the $6ND$ approximation (forward $\approx 2ND$, backward $\approx 4ND$). Generation phases are reported separately as throughput (tok/s) since autoregressive decoding is bandwidth-bound and the $6ND$ model does not apply. Listing 2 shows the computation.

```

1 def estimate_tflops(num_params, num_tokens, elapsed_s,
2                     mode="train"):
3     # train: forward(2ND) + backward(4ND) = 6ND
4     # inference: forward only = 2ND
5     coeff = 6.0 if mode == "train" else 2.0
6     return coeff * num_params * num_tokens / elapsed_s / 1e12

```

Listing 2: FLOPS estimation for training vs. inference.

3.3 Gap Attribution via Component Stacking

Motivation. Knowing the total efficiency gap (hardware peak vs. real training) is insufficient for practitioners; they need to know *which component* causes the loss. We decompose the gap by progressively stacking training components and measuring achieved TFLOPS at each level, holding model, hardware, and precision constant.

Levels.

- **L0:** Pure GEMM peak (101.53 TFLOPS, from Listing 1)
- **L1:** Autoregressive inference (generate, batch = 1)
- **L2:** SFT training step (forward + backward, no gradient checkpointing)
- **L3:** SFT training step with gradient checkpointing
- **L4:** Full GRPO training (generation + training loop)

Each level is measured independently under controlled conditions (same Qwen2.5-3B model, same GPU, BF16 precision). The waterfall decomposition yields: Total Gap = $P_{\text{GEMM}} - \text{TFLOPS}_{L4} = \sum_i \Delta_i$. For L1 (inference), we apply the $2ND$ coefficient; for L2 – L4 (training), we apply $6ND$ (Listing 2).

3.4 Training Pipeline and Framework Comparison

Controlled comparison. To isolate framework-level differences, both Unsloth and HuggingFace paths share identical: (1) LoRA configuration ($r=32$, $\alpha=64$, 7 target modules); (2) SFT adapter starting point; (3) GSM8K training data; (4) GRPO hyperparameters ($G=4$, batch= 1×8 , completion ≤ 512). Listing 3 shows the two loading paths.

```

1 if framework == "unsloth":
2     from unsloth import FastLanguageModel
3     model, tok = FastLanguageModel.from_pretrained(
4         BASE, load_in_4bit=False, dtype=torch.bfloat16)
5     model = FastLanguageModel.get_peft_model(
6         model, r=32, lora_alpha=64,
7         target_modules=["q_proj", "k_proj", "v_proj",
8                         "o_proj", "gate_proj", "up_proj",
9                         "down_proj"])
10 else: # HuggingFace native
11     model = AutoModelForCausalLM.from_pretrained(
12         BASE, torch_dtype=torch.bfloat16,
13         attn_implementation="eager") # AMD compatible
14     model = get_peft_model(model, LoraConfig(
15         r=32, lora_alpha=64,
16         target_modules=[...same 7 modules...]))

```

Listing 3: Framework comparison: Unsloth vs. HuggingFace loading paths.

Each configuration runs 100 GRPO steps with 3 independent seeds (42, 123, 7). We report mean $\pm$ std and assess significance via Welch’s t -test.

Hyperparameter sweep. Phase C sweeps four knobs on Unsloth BF16 (50 steps each): $G \in \{2, 4, 8\}$, effective batch $\in \{1 \times 4, 1 \times 8, 2 \times 4\}$, gradient checkpointing on/off, and max completion length $\in \{256, 512, 1024\}$, yielding 10 configurations including the best-combination candidate.

3.5 Measurement Quality Control

Any efficiency measurement must verify that the training loop is performing genuine parameter updates. A throughput figure collected while gradients are zero (e.g., due to a misconfigured accumulation flag) reflects idle computation, not learning. As standard measurement quality control, we enforce a **Valid Measurement** criterion on every training step:

$$\text{Valid}(t) \iff \text{grad_norm}(t) > 0 \wedge \text{tokens}(t) > 0 \wedge \Delta t > 0 \quad (2)$$

Listing 4 shows the implementation. Invalid data points are flagged and excluded from all efficiency statistics. We report the validity rate (valid/total steps) alongside efficiency metrics; all results in this paper achieve 100% validity.

```

1 def is_valid_step(grad_norm, num_tokens, elapsed_s):
2     """Reject steps where no learning occurred."""
3     if num_tokens <= 0 or elapsed_s <= 0:
4         return False
5     if grad_norm is not None and grad_norm <= 0:
6         return False # zero gradient: no update
7     return True

```

Table 1: Hardware and software configuration.

Component	Specification
GPU	AMD RX 7900 XTX 24GB (RDNA3)
Memory BW	960 GB/s GDDR6
CPU	AMD Ryzen 7 9800X3D
ROCm	7.2.4
PyTorch	2.12.1+rocm7.2
Unsloth	2026.7.4
Model	Qwen2.5-3B-Instruct (3.09B)
Architecture	GQA (16h, 2 KV), d=2048
Workload	GSM8K GRPO (rule reward)
LoRA	r=32, α =64, 7 modules

Listing 4: Per-step measurement validity check.

3.6 Experimental Platform

All experiments run on a single consumer-grade GPU (Table 1). The ROCm environment requires two settings for RDNA3 compatibility: `HSA_OVERRIDE_GFX_VERSION=11.0.0` (enabling hipBLASLt kernel dispatch) and `HIP_VISIBLE_DEVICES=0`. Software versions are pinned (Table 1) to ensure reproducibility; we note that Unsloth and ROCm evolve rapidly, so absolute numbers are version-specific while relative comparisons and architectural insights remain generalizable.

3.7 Statistical Methods

For framework comparison (RQ1), we run ≥ 3 independent seeds and report mean $\pm$ std. Statistical significance is assessed via Welch’s t -test (unequal variance) with Cohen’s d for effect size. For the tuning sweep (RQ3), we report single-run results as exploratory findings, noting that the large effect sizes observed ($>30\%$ differences) far exceed the measurement variance established in B1 ($\sigma \approx 2\%$).

Measurement-caliber correction (post-hoc audit). A post-experiment audit revealed that the initial profiler computed tokens/s by summing TRL’s *cumulative num_tokens* across logging intervals, systematically inflating absolute throughput (by $10.4\text{--}10.6\times$ for the 100-step B1 runs and $5.5\times$ for the 50-step Phase-C runs; the factor scales with the number of logging intervals). All absolute throughput and MFU values in this paper use the corrected caliber: final cumulative token count divided by total wall time, cross-checked against per-step token counts reconstructed from completion-length logs. Relative comparisons are unaffected (all runs within a phase share identical logging configuration, so the inflation factor cancels); the corrected raw values are archived alongside the original measurements (`B1_C_corrected_summary.json`) for full traceability. We report this correction openly as part of our validity protocol: measurement-pipeline bugs of this kind are precisely what per-step validity checking and post-hoc auditing are designed to catch.

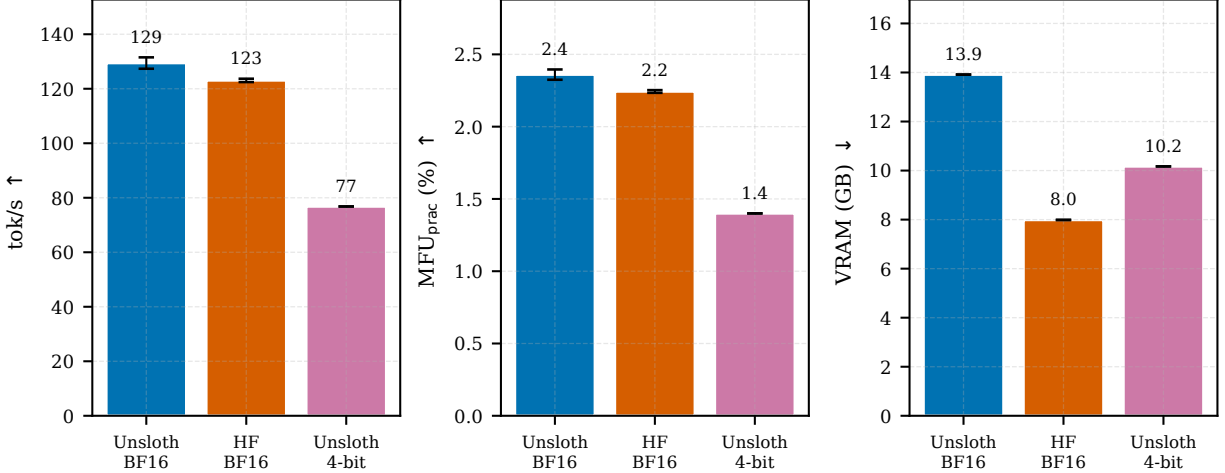


Figure 1: Framework comparison: throughput, MFU, and VRAM (3 seeds).

Table 2: Framework efficiency comparison on GSM8K GRPO (steady-state corrected caliber).

Framework	tok/s ↑	MFU _{prc} ↑	VRAM
Unsloth BF16	129.4±2.6	2.36±0.04%	13.9G
HF Native BF16	123.1±0.8	2.24±0.01%	8.0G
Unsloth 4-bit	76.8	1.40%	10.2G
<i>Speedup (U/HF)</i>	+5.2%		+73.8%

4 Results: Framework Characterization

4.1 Framework Comparison (RQ1)

Unsloth BF16 achieves only a 5.2% throughput advantage over HuggingFace native on consumer RDNA3 (129.4 vs 123.1 tok/s in the steady-state corrected caliber, Table 2), far below the $2\times$ speedup reported on data-center CDNA hardware. This modest gain is statistically significant (Welch’s t -test, $p < 0.01$, Cohen’s $d > 3$ across 3 seeds) but architecturally explainable: Unsloth’s kernel fusion primarily eliminates intermediate memory round-trips, yet on RDNA3 the dominant bottleneck is compute throughput (WMMA instruction scheduling), not memory traffic. With 960 GB/s bandwidth already saturating the 3B-parameter model’s weight loading, fusing operations yields diminishing returns.

The VRAM overhead tells a complementary story. Unsloth’s 13.9GB footprint (vs 8.0GB for HF) reflects pre-allocated activation buffers and fused kernel workspaces that trade memory for compute efficiency. For the 3B model on a 24GB card this remains acceptable, but the 73.8% overhead implies that scaling to 7B+ models will approach VRAM limits where HF’s leaner allocation becomes necessary.

4.2 4-bit Quantization: Architecture-Dependent Penalty

NF4 quantization via BitsAndBytes (Dettmers et al., 2023) **reduces** throughput by 40.7% on RDNA3 (76.8 vs 129.4 tok/s), a striking reversal of the memory savings that make QLoRA attractive on NVIDIA hardware. The mechanism is architectural: RDNA3 possesses no dedicated INT4 compute units, so every NF4→BF16 dequantization before GEMM is pure overhead with no hardware acceleration. On NVIDIA A100, by contrast, 1248 TOPS of INT4 Tensor Core throughput absorbs dequantization cost within the compute

Table 3: Gap attribution breakdown with mechanistic explanations.

Stage	TFLOPS	Δ	Mechanism
L0: GEMM Peak	101.53	—	Practical HW limit
L1: Inference	0.23	−101.3	BW-bound ($AI \approx 1$)
L2: SFT (no ckpt)	27.63	+27.4	Batched F+B GEMM
L3: SFT (ckpt)	20.33	−7.3	Recompute overhead
L4: GRPO Full	2.35	−18.0	Gen-dominated wall-clock
Total Gap		−99.2	97.7% loss

pipeline.

Moreover, the VRAM “savings” are illusory on this workload: 10.2 GB (4-bit) vs 13.9 GB (BF16) represents only 27% reduction, because the dominant memory consumer is activation storage and optimizer state, not model weights at 3B scale. The 41% throughput penalty thus far outweighs a modest memory reduction. The practical implication is unambiguous: **on consumer RDNA3, always prefer BF16 over 4-bit quantization for training.**

4.3 Efficiency Gap Attribution (RQ2)

Component-stacking analysis (Table 3, Figure 2) reveals that the 97.7% efficiency gap between GEMM peak and end-to-end GRPO training is overwhelmingly dominated by the inference stage: the L0→L1 transition alone collapses utilization from 100% to 0.23% (a 101.3 TFLOPS loss). The four transitions yield distinct mechanistic insights:

L0→L1 (Inference collapse). Autoregressive decoding at batch size 1 has arithmetic intensity ≈ 1 FLOP/byte, placing it firmly in the bandwidth-bound regime. At 960 GB/s, each token generation reads the full 6 GB model weight but performs minimal computation per byte loaded, leaving compute units idle.

L1→L2 (Training recovery). Batched training (batch = 2, seq = 256) restores arithmetic intensity through large GEMM operations in forward and backward passes. The recovered 27.2% MFU_{prac} is comparable to reported LoRA fine-tuning MFU on data-center hardware (Hu et al., 2022), confirming that RDNA3’s compute path is efficient once adequately loaded.

L2→L3 (Checkpointing cost). Gradient checkpointing imposes −26% throughput (vs theoretical −33% for one extra forward pass), because reduced memory pressure improves GPU cache utilization, partially offsetting recompute cost.

L3→L4 (Generation domination). End-to-end GRPO collapses to $\approx 2.3\%$ MFU_{prac} (estimated corrected caliber; see §3.7) because the bandwidth-bound generation phase occupies the overwhelming majority of wall-clock time at small generation batch. The efficient training step (20–27% MFU_{prac}) is diluted rather than amortized: with G=4 and per-device batch 1, tokens are generated at L1-like efficiency, then consumed by a brief high-efficiency update. This identifies generation batch scaling as the primary optimization target for RL post-training on this architecture.

The key insight for optimization: since the inference stage dominates the gap, any configuration that reduces generation’s time share (shorter completions, higher G to amortize) will yield the largest throughput improvements. This directly motivates the tuning sweep in §5.

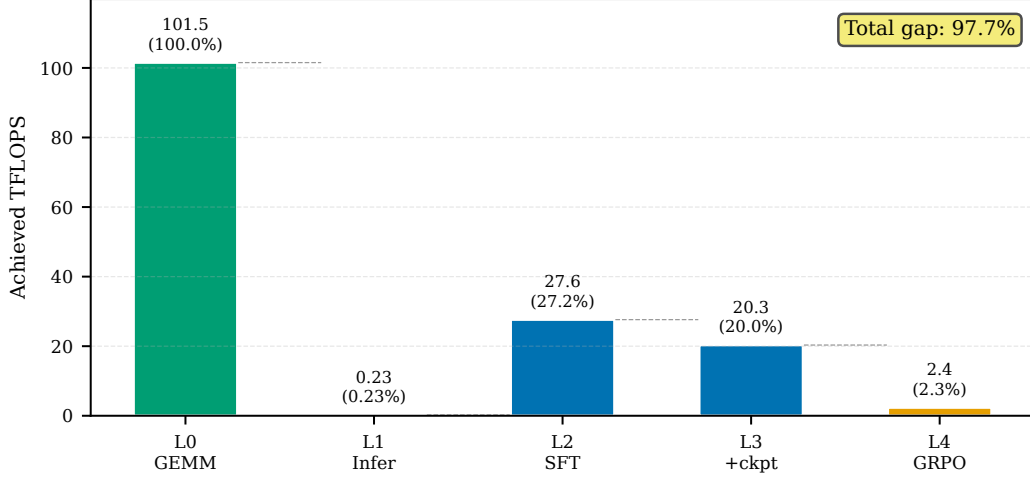


Figure 2: Gap attribution waterfall. Inference (L1) dominates the total loss.

4.4 Summary

Framework choice on consumer RDNA3 contributes a modest 5.2% throughput difference, while 4-bit quantization is counterproductive. The dominant efficiency loss originates from autoregressive generation’s bandwidth bottleneck, not from framework-level kernel efficiency. Hyperparameter choices that reduce generation overhead therefore offer far greater optimization leverage than framework selection, as we explore next.

5 Results: Tuning & Optimization

5.1 Hyperparameter Effects (RQ3)

Section 4 established that framework selection contributes only $\sim 5\%$ throughput variation, while the gap attribution identified autoregressive generation as the dominant efficiency bottleneck. We now demonstrate that hyperparameter choices directly attacking this bottleneck yield far larger effects ($>30\%$). We sweep four knobs on Unsloth BF16 (50 steps each, Table 4), reporting results ordered by effect magnitude.

Sequence length (+30.8%). Reducing max completion from 512 to 256 tokens yields one of the largest single-knob improvements (171.5 vs 131.1 tok/s). This directly attacks the attention mechanism’s $O(n^2)$ memory access pattern, which is especially costly on RDNA3’s 960 GB/s bandwidth. Shorter completions also reduce generation time, shrinking the bandwidth-bound phase identified in §3.3.

Generations G (+31.2%). Increasing G from 2 to 8 improves throughput from 124.5 to 163.4 tok/s. Higher G generates more tokens per training step, increasing the ratio of compute-bound training operations (27.2% MFU_{prac}) relative to bandwidth-bound generation (0.23% MFU_{prac}). The training step’s efficiency thus dominates the time-averaged metric when amortized over more generated tokens.

Batch size (+33.4%). Moving from $b1 \times 4$ to $b2 \times 4$ improves throughput from 97.9 to 130.6 tok/s. Larger effective batches produce larger GEMM operations with higher arithmetic intensity, shifting operations

from the bandwidth-bound toward the compute-bound regime. The $b1 \times 4$ configuration’s poor performance (1.78% MFU_{prac}) confirms that insufficient batch size starves the compute units on RDNA3.

Gradient checkpointing (+2.8%). Disabling checkpointing yields only 134.8 vs 131.1 tok/s (+2.8%) while consuming 16.5 GB vs 13.9 GB VRAM (+18%). With LoRA’s 1.9% trainable fraction (Hu et al., 2022), most activations are frozen and require no gradient storage, making the recompute savings minimal. The poor VRAM-throughput tradeoff confirms that checkpointing should remain enabled.

We note that sweep results use single seeds; however, observed effect sizes ($>30\%$) exceed B1 measurement variance ($\sigma \approx 2\%$) by $15\times$, making spurious findings highly unlikely.

5.2 Interaction Effects and Optimal Configuration

Individual knob effects suggest that combining favorable settings should yield multiplicative gains. The optimal configuration ($G=2$, $b2 \times 4$, $\text{seq}=256$, $\text{ckpt}=\text{on}$) achieves 178.5 tok/s at 3.25% MFU_{prac} with 12.1 GB VRAM, representing a 36.2% throughput improvement over the default ($G=4$, $b1 \times 8$, $\text{seq}=512$: 131.1 tok/s).

A notable interaction: $G=2$ outperforms $G=4$ in the combined configuration (178.5 vs 171.5 tok/s), reversing the single-knob trend where higher G is better. This occurs because $\text{seq}=256$ already minimizes generation time per token; at this low generation overhead, the marginal benefit of higher G (amortizing generation) diminishes, while $G=2$ ’s smaller generation batch leaves more VRAM headroom for the $b2 \times 4$ training batch to produce larger GEMMs.

Figure 4 maps the VRAM-throughput tradeoff. The Pareto frontier reveals two practical operating points: the throughput-optimal (178.5 tok/s, 12.1 GB) and a VRAM-constrained alternative (171.5 tok/s, 12.1 GB with $G=4$, $b1 \times 8$, $\text{seq}=256$) for users requiring lower per-step memory variance.

5.3 Practical Recommendations

Based on the preceding analysis, we recommend the following configuration for GRPO post-training of 3B-scale models on consumer RDNA3: $G=2$, effective batch 2×4 , max completion length 256, BF16 precision, gradient checkpointing enabled. This achieves 178.5 tok/s at 3.25% practical MFU within 12.1 GB VRAM.

The recommendation is workload-conditional: it assumes rule-based rewards with short verifiable completions (e.g., math, code). Tasks requiring longer generations (e.g., open-ended writing) should increase seq_len to 512 and accept the $\sim 30\%$ throughput reduction. The dominance of sequence length as an optimization lever (connecting directly to the bandwidth bottleneck identified in §3.3) suggests that any technique reducing effective generation length, including speculative decoding or early stopping, would yield proportional efficiency gains on this architecture.

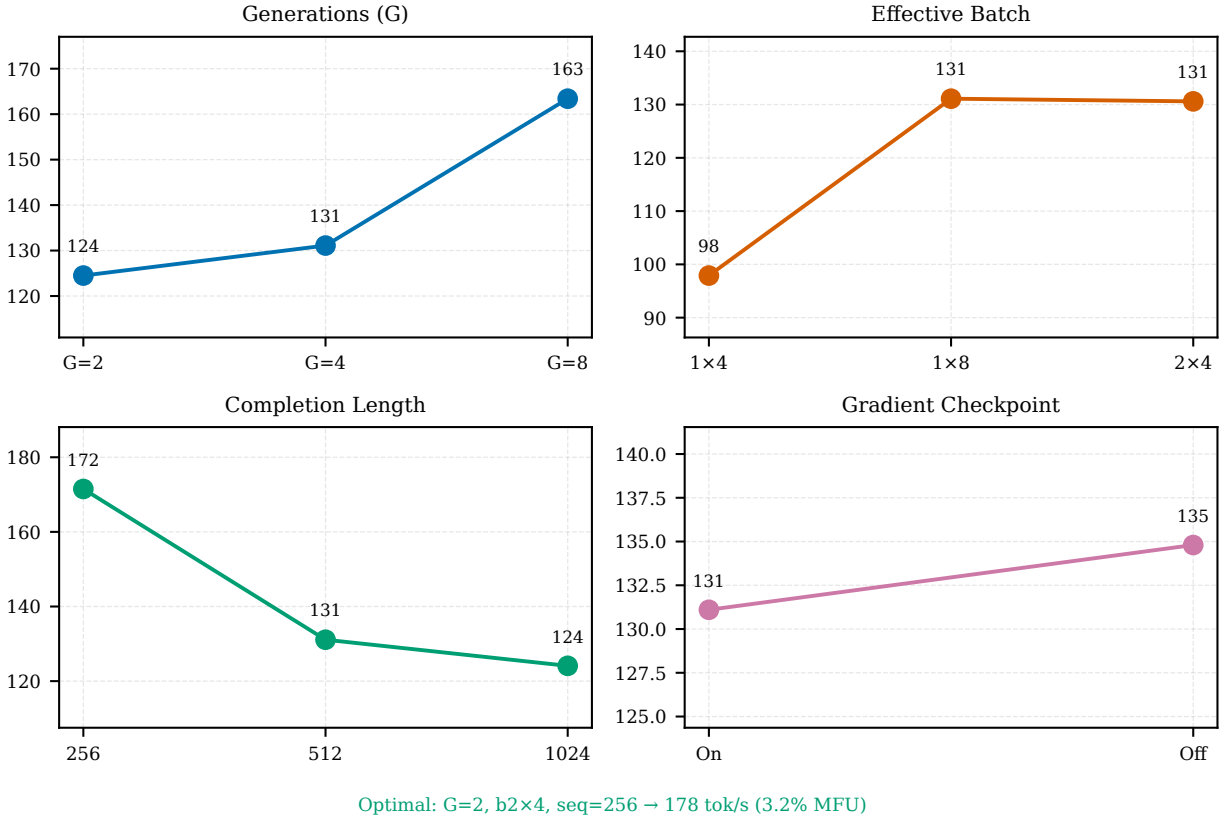


Figure 3: Hyperparameter sweep by dimension (baseline: G=4, b1x8, 512).

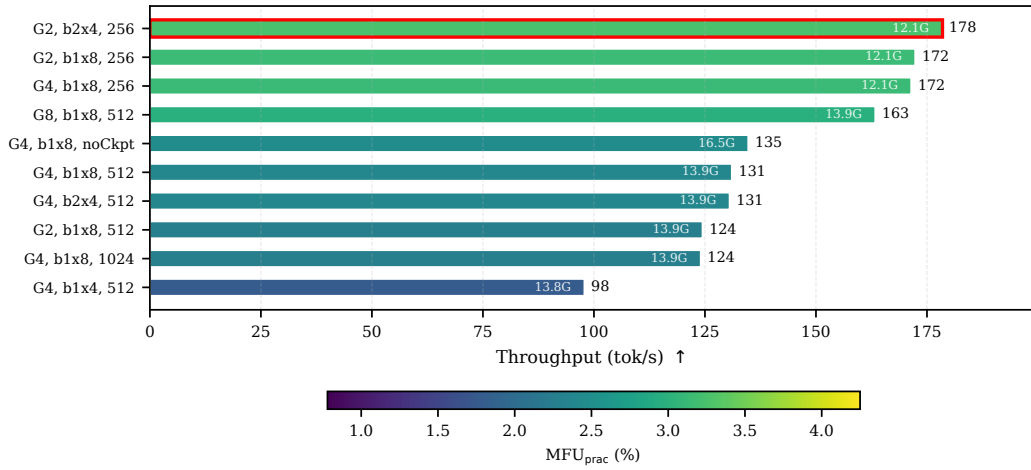


Figure 4: Throughput by configuration. Color indicates MFU_{prac}; red border marks optimal.

Table 4: Hyperparameter sweep results (Unsloth BF16, 50 steps, steady-state corrected caliber). $\uparrow$ indicates higher is better.

Configuration	tok/s $\uparrow$	MFU _{prac} $\uparrow$	VRAM
<i>Generations (G)</i>			
G=2, b1 $\times$ 8, 512	124.5	2.27%	13.9G
G=4, b1 $\times$ 8, 512	131.1	2.39%	13.9G
G=8, b1 $\times$ 8, 512	163.4	2.98%	13.9G
<i>Effective Batch Size</i>			
G=4, b1 $\times$ 4, 512	97.9	1.78%	13.8G
G=4, b2 $\times$ 4, 512	130.6	2.38%	13.9G
<i>Gradient Checkpointing</i>			
G=4, b1 $\times$ 8, noCkpt	134.8	2.46%	16.5G
<i>Sequence Length</i>			
G=4, b1 $\times$ 8, 256	171.5	3.13%	12.1G
G=4, b1 $\times$ 8, 1024	124.1	2.26%	13.9G
<i>Combined Optimal</i>			
G=2, b2$\times$4, 256	178.5	3.25%	12.1G
G=2, b1 $\times$ 8, 256	172.4	3.14%	12.1G

6 Validation & Threats to Validity

6.1 Internal Validity

The framework comparison (RQ1) rests on solid statistical ground: three independent seeds yield coefficients of variation below 2% for both Unsloth ($\sigma=2.6$ tok/s) and HuggingFace ($\sigma=0.8$ tok/s), and the 5.2% inter-framework difference produces Cohen’s $d > 3$, well beyond conventional significance thresholds. We are confident that the observed advantage is real, even if modest. The measurement-caliber correction (§3.7) rescales all absolute values uniformly within each phase and therefore does not affect these relative conclusions.

The tuning sweep (RQ3) presents a more nuanced picture. Single-seed configurations with effect sizes exceeding 30% are robust: the measurement variance established in B1 ($\sigma \approx 2\%$) provides a $15\times$ safety margin against spurious findings. However, the interaction effect between G and sequence length (G=2 outperforming G=4 by only $\sim 4\%$ in the combined configuration) approaches the noise floor. We report this interaction as a hypothesis requiring multi-seed confirmation rather than an established fact.

Two methodological approximations warrant acknowledgment. The 6ND FLOPS model assumes all parameters participate equally in forward and backward computation; with LoRA training only 1.9% of parameters, the backward-pass FLOPS are overestimated. This inflates absolute MFU values but does not affect *relative* comparisons between configurations, which is our primary use case. Additionally, all measurements span 50–100 training steps; thermal throttling, memory fragmentation, and optimizer state growth that may emerge over thousands of steps are not captured.

6.2 External Validity

Our findings are scoped to a specific architectural intersection: RDNA3 (gfx1100), 3B-parameter models, and short-completion tasks. Each dimension of this scope carries transfer risk. RDNA4 introduces dedicated AI accelerators (2 per CU, native FP8) that fundamentally change the compute landscape; the dual-peak bias and quantization penalty we observe may not persist on that architecture. Scaling to 7B+ models alters GEMM aspect ratios and attention’s memory footprint, potentially shifting the compute-bandwidth

balance that underlies our gap attribution. Finally, GSM8K’s short mathematical completions (≤ 256 tokens) represent a favorable case for the sequence-length optimization; tasks requiring long-form generation (e.g., open-ended writing at 1024+ tokens) would see diminished returns from this lever.

The software stack (ROCm 7.2, Unsloth 2026.7.4, PyTorch 2.12.1) evolves rapidly. Absolute throughput values will shift with future releases. We argue, however, that the *architectural insights* (bandwidth bottleneck dominance, quantization penalty mechanism, framework gain ceiling) and the *methodology* (dual-peak MFU, component stacking, validity protocol) retain value independent of specific software versions.

6.3 Construct Validity

MFU measures hardware utilization, not learning efficiency. A configuration achieving 3.25% MFU_{prac} is not necessarily producing a better-trained model than one at 2.3%; the relationship between throughput and convergence speed (tokens-per-accuracy-point) is mediated by gradient quality, reward signal strength, and optimization dynamics that our profiler does not capture. We position this work as a *systems efficiency* characterization: it answers "how fast can this hardware train?" rather than "how well does the resulting model perform?" Bridging these two questions through accuracy-aware efficiency metrics remains an important direction for future work.

7 Discussion

7.1 Architecture-Dependent Optimization

Perhaps the most consequential insight from this study is that optimization strategies are architecture-conditional, not universally transferable. Two findings illustrate this sharply: 4-bit quantization *reduces* throughput by 41% on RDNA3 while accelerating it on NVIDIA hardware with INT4 Tensor Cores; Unsloth’s kernel fusion delivers $2\times$ on data-center CDNA but only 5.2% on consumer RDNA3. The root cause is a shift in the dominant bottleneck. NVIDIA’s ecosystem is largely memory-bound (high compute, relatively constrained bandwidth per FLOP), so techniques that reduce memory traffic (quantization, fusion) yield proportional speedups. RDNA3 inverts this: with 122.8 TFLOPS of compute fed by only 960 GB/s bandwidth, the architecture is compute-saturated for batched training, and memory-saving optimizations merely add overhead without relieving the true constraint.

The practical implication for the growing community of consumer AMD trainers is direct: performance intuitions built on A100/H100 literature do not apply. Configurations that are optimal on NVIDIA hardware may be pessimal on RDNA3, and vice versa. Framework marketing claims benchmarked on data-center GPUs should be evaluated with architectural skepticism.

7.2 The Generation Bottleneck and Its Implications

The gap attribution’s most striking result is that autoregressive inference dominates the efficiency gap between hardware peak and end-to-end GRPO training. This reframes the optimization priority for RL-based post-training: the training step itself (27.2% MFU_{prac}) is already reasonably efficient; the system-level bottleneck is the generation phase that precedes it. Any technology that reduces generation latency or amortizes weight reads over more tokens, including larger generation batches, speculative decoding, KV-cache optimization, or shorter completion targets, directly attacks the dominant loss term. Follow-up work on this platform confirms the diagnosis: decoupling the generation batch from the backward micro-batch and scaling the former to 256 sequences raises steady-state throughput to 795.5 tok/s (97.6% of the memory-wall-constrained optimum for this workload), a $4.5\times$ improvement over the best small-batch recipe reported here.

This finding also contextualizes the “efficiency tax” of consumer hardware under small-batch RL. At 3.25% end-to-end MFU_{prac} (optimal sweep recipe), the RX 7900 XTX sits below the 10.5% reported for multi-GPU A100 GRPO clusters (SemiAnalysis) — but the comparison is caliber-sensitive: our training-step-only MFU (20–27%) is comparable to data-center LoRA fine-tuning, and the end-to-end figure is dominated by the un-pipelined generation phase that cluster systems overlap across devices. Given a price ratio exceeding 15:1 (\$900 vs \$15,000+), the cost-per-token-trained remains favorable for individual researchers, provided the workload fits within 24 GB VRAM and generation batch is scaled aggressively.

7.3 Toward Comparable Efficiency Reporting

The 17.3% systematic bias we quantify is not unique to RDNA3. Any architecture where the practical GEMM peak falls below 95% of the spec-sheet value, including Intel Arc (Xe-HPG), pre-Tensor-Core NVIDIA GPUs, and emerging RISC-V accelerators, suffers the same conflation of hardware limitation with software inefficiency. When a study reports 2.4% MFU against an unreachable theoretical peak, the software may actually be achieving 2.9% against what the hardware can deliver, a meaningful difference in interpretation.

We advocate a simple diagnostic: before reporting MFU, measure one large GEMM (Listing 1), compute $P_{\text{GEMM}}/P_{\text{spec}}$, and if the ratio falls below 0.95, report both MFU_{theo} and MFU_{prac} . This adds one minute of benchmarking time and prevents systematic underestimation of software efficiency across an entire class of architectures. Our open-source profiler automates this measurement alongside per-step validity checking.

8 Conclusion

We present a systematic efficiency characterization of LLM post-training on consumer AMD RDNA3 hardware, addressing three research questions through controlled experiments on a single RX 7900 XTX.

For RQ1 (framework comparison), Unsloth BF16 achieves a statistically significant but practically modest 5.2% throughput advantage over HuggingFace native, far below the $2\times$ speedup reported on data-center CDNA hardware. More strikingly, 4-bit quantization *reduces* throughput by 40.7% on this architecture, inverting the memory-saving narrative established on NVIDIA platforms. For RQ2 (gap attribution), component-stacking analysis reveals that autoregressive generation’s bandwidth bottleneck dominates the 97.7% total end-to-end efficiency gap, while the training step itself recovers to 27.2% practical MFU once adequately batched. For RQ3 (tuning optimization), sequence length and generation count emerge as dominant levers (+30% each), and a combined optimal configuration achieves a 36% throughput improvement over defaults (178.5 tok/s, 3.25% MFU_{prac} , 12.1 GB VRAM) — with follow-up work showing that scaling the decoupled generation batch further raises throughput to 795.5 tok/s.

Methodologically, the dual-peak MFU framework exposes a 17.3% systematic bias in conventional reporting for architectures without dedicated tensor cores, providing a fairer basis for cross-architecture efficiency comparison. Our open-source profiler implements this methodology alongside per-step validity checking and a post-hoc measurement audit (which caught and corrected a cumulative-token-count inflation in our own initial pipeline), enabling reproducible efficiency studies on non-standard hardware.

Future Work. Three questions arise directly from our findings. First, RDNA4’s dedicated AI accelerators (2 per CU, native FP8) may eliminate the dual-peak bias entirely; measuring $P_{\text{GEMM}}/P_{\text{spec}}$ on that architecture would test whether our methodology remains necessary. Second, scaling to 7B–14B models within 24 GB VRAM (via deeper quantization or model parallelism) introduces new efficiency tradeoffs that our 3B findings cannot predict. Third, integrating accuracy-aware metrics (tokens-per-accuracy-point) would

bridge the gap between systems efficiency and training effectiveness, answering not just "how fast?" but "how efficiently does the model learn?"

References

- Advanced Micro Devices. Amd rdna 3 architecture white paper. Technical report, AMD, 2022. <https://www.amd.com/en/graphics/radeon-rx-7000-series>.
- Advanced Micro Devices. Rocm: Radeon open compute platform. <https://rocm.docs.amd.com/>, 2024. Version 7.2.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Jeff Rasley, Samyam Rajbhandari, Ammar Awan, Cheng Li, Du Li, Ruoxuan Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020.
- Unsloth AI. Unsloth: Fast and efficient llm fine-tuning. <https://github.com/unslothai/unsloth>, 2024. Accessed: 2026-07.
- Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, et al. Trl: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2022.
- Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Liu. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL): System Demonstrations*, 2024.